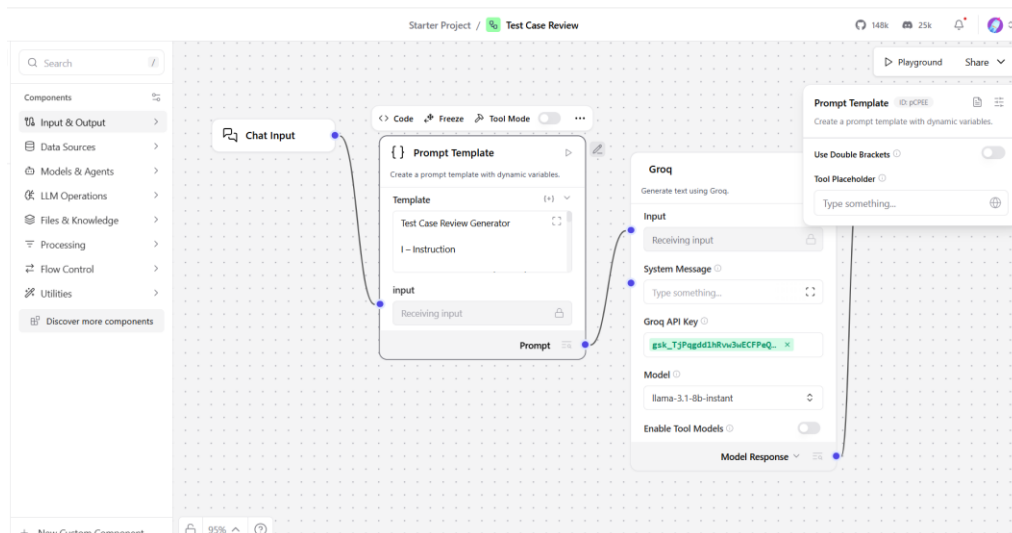




Langflow json



Test Case
Review.json

Scenario 1:

Provided valid test case

Review the below testcase**Merchandise Plan – Functional Test Cases (Retail Domain)**

TC-ID	Priority	Test Description	Pre-Requisite	Test Steps	Test Data	Expected Result	Actual Result
TC-MerchPlan-001	P1	Verify that a Merchandise Plan can be created with all mandatory fields	Application is up, user has "Create Merchandise Plan" role, sample product & location data exist	1. Login as Retail Analyst 2. Navigate to "Merchandise Plans" → "Create New" 3. Enter Plan Name, Calendar, Product, Location, Start/End dates 4. Click "Save"	PlanName: "Q3 2026 Plan" Calendar: "2026 Q3" Product: "SKU-12345" Location: "Store-001"	Plan is created and appears in the list with status "Draft"	–
TC-MerchPlan-002	P1	Verify that the system prevents creation of a plan with duplicate name	Same as above	1. Create plan "Q3 2026 Plan" (as in TC-001) 2. Attempt to create another plan with same name Same data as TC-001		Error message "Plan name already exists" and plan not created	–
TC-MerchPlan-003	P1	Verify that a plan can be published after seeding data from previous year	Plan created (TC-001)	1. Open plan "Q3 2026 Plan" 2. Click "Seed from Previous Year" 3. Confirm seed 4. Update WP sales details 5. Click "Publish"	WP Sales R: 245,362	Plan status changes to "Published" and audit log shows publish date	–
TC-MerchPlan-004	P1	Verify that a published plan cannot be edited	Published plan (TC-003)	1. Open plan "Q3 2026 Plan" 2. Attempt to edit any field		Edit controls disabled; message "Plan is published – editing not allowed"	–
TC-MerchPlan-005	P2	Verify that a plan can be duplicated with a new name	Existing plan (TC-001)	1.			



Result Screenshot:

✓ Finished in 6.0s

Test-Case Review – Merchandise Plan (Retail Domain)

1. Issues Identified

#	Issue	Affected TC(s)	Impact
1	Inconsistent or missing pre-conditions – some cases assume data created in earlier tests without explicitly stating that the prerequisite test must have run successfully (e.g., TC-MerchPlan-003, TC-MerchPlan-004, TC-MerchPlan-010-013).	3,4, 10-13	Test execution can become flaky if the dependent test fails or is skipped.
2	No clean-up / post-condition steps – after creating, publishing, duplicating or deleting plans the environment is left in a changed state. This can affect subsequent runs.	1-7, 10-13, 15	Test suite is not repeatable; may require manual reset.
3	Test data is hard-coded and not parameterised – values such as “Q3 2026 Plan”, SKU-12345, dates, etc., are embedded directly in the steps. This reduces re-usability and makes maintenance harder when data changes.	All	Limits scalability and data-driven execution.

1. Issues Identified

	such as “Q3 2026 Plan”, SKU-12345, dates, etc., are embedded directly in the steps. This reduces re-usability and makes maintenance harder when data changes.		data-driven execution.
4	Ambiguous verification points – several “Expected Result” columns describe high-level outcomes but omit concrete UI elements, API responses, or database checks (e.g., TC-MerchPlan-001 “appears in the list”, TC-MerchPlan-012 “Total WP Sales = 245,362”).	1, 12, 13, 14	May lead to subjective interpretation and missed defects.
5	Missing negative-scenario coverage – only a few negative cases are present (duplicate name, duplicate product, delete published plan). Other important validations are absent, such as: • Invalid product SKU • Unauthorized user trying to delete a draft • Attempt to publish without required fields filled.	–	Reduces confidence in robustness of the Merchandise-Plan module.
6	Role-based access test limited to a single role – TC-MerchPlan-015 checks only the “Viewer” role. It does not verify that a “Merch Analyst” <i>can</i> publish, nor does it test other roles (e.g., “Admin”, “Store Manager”).	15	Incomplete RBAC validation.
7	Date handling edge cases not covered – only a simple start-date > end-date check (TC-MerchPlan-009). No tests for leap-year dates, timezone differences, or boundary dates (e.g., same-day start/end).	9	Potential date-related bugs remain unchecked.
8	Audit-trail verification is superficial – TC-MerchPlan-014 only checks creation log. Updates, deletions, and publish actions are not verified.	14	Audit compliance gaps may go unnoticed.
9	Performance / concurrency not addressed – No test validates system behaviour when multiple users edit/duplicate plans simultaneously, or when a large number of products are added to a plan.	–	May miss scalability or locking issues.
10	Inconsistent naming / formatting – Some TC IDs use hyphens (TC-MerchPlan-001) while others use underscores in description; step numbering mixes HTML ` ` tags with plain text. This hampers readability and automated parsing.	All	Reduces clarity and tool-compatibility.

10	Inconsistent naming / formatting – Some TC IDs use hyphens (TC-MerchPlan-001) while others use underscores in description; step numbering mixes HTML ` ` tags with plain text. This hampers readability and automated parsing.	All	Reduces clarity and tool-compatibility.
11	Missing environment/context details – No column for “Test Environment” (e.g., DEV, QA, PROD) or “Browser/Device” for UI tests.	All	Execution may vary across environments without traceability.
12	No explicit acceptance criteria for “Draft” vs “Published” status – Expected results mention status change but do not verify that the status field is read-only, colour-coded, or reflected in API payloads.	3-4, 6-7	Ambiguity in what constitutes a successful status transition.
13	Lack of traceability to requirements – No reference to functional requirement IDs or user stories, making impact analysis difficult.	All	Gaps in requirement-coverage matrix.
14	“Actual Result” column left blank – While understandable for a design-time document, the template should indicate that this column will be populated during execution, or be omitted from the design artefact.	All	Minor but can cause confusion for reviewers.

2. Suggestions for Improvement

#	Recommendation	Rationale / How to Apply
A	Add explicit pre-condition and post-condition sections for each test case.** e.g., “Pre-condition: Plan Q3 2026 Plan exists in Draft status.” Post-condition: Delete the created plan to restore baseline.	Guarantees test independence and repeatability.
B	Parameterise test data using a data-driven approach (CSV, Excel, JSON, or test-management tool variables). Replace hard-coded values with placeholders (e.g., `\${PlanName}`, `\${SKU}`).	Simplifies maintenance, enables multiple data sets, and supports CI execution.
C	Detail verification steps: specify UI element identifiers (label, tooltip, CSS selector), API response fields, or database queries. Example: “Verify that the plan appears in the grid with column Plan Name = Q3 2026 Plan and Status = Draft.”	Removes ambiguity, supports automated validation, and ensures consistent defect reporting.
D	Expand negative-scenario coverage: • Attempt to create a plan with an invalid SKU. • Try to publish a plan while missing mandatory fields. • Delete a Draft plan with a user lacking delete permission.	Improves confidence that validation logic is comprehensive.

2. Suggestions for Improvement

E	Broaden RBAC tests: include at least three roles – Viewer (cannot publish/delete), Merch Analyst (can publish, edit Draft), and Admin (full rights). Verify both allowed and denied actions for each role.	Confirms that role-based security is correctly enforced across the matrix.
F	Add date-edge-case tests: • Start = End (date same day). • Leap-year date (Feb 29). • Dates across DST changes or different time zones if applicable.	Prevents subtle bugs related to calendar handling.
G	Enhance audit-trail verification: create separate tests for creation, edit, publish, duplicate, and delete actions, checking timestamps, user IDs, and action types in the audit log.	Supports compliance and traceability requirements.
H	Introduce performance / concurrency tests (optional for functional suite, but valuable): • Add 500 products to a plan and verify response time • Simultaneous duplicate actions by two users and verify locking behavior.	Detects scalability and race-condition issues early.
I	Standardise naming and formatting: • Use a consistent TC-ID pattern (e.g., 'TC_MP_001'). • List steps as numbered bullet points without HTML tags. • Keep columns aligned and avoid mixed markup.	Improves readability and enables import into test-management tools.
J	Add “Environment” and “Browser/Device” columns (or a global test-environment section) to capture where the test should run.	Provides traceability across environments and helps reproduce defects.
K	Link each test case to requirement/user-story IDs (e.g., 'REQ-RT-102').	Facilitates coverage analysis and impact assessment for changes.
L	Clarify the purpose of the “Actual Result” column in the design document – either remove it (to be filled during execution) or add a note such as “To be populated after test run”.	Prevents confusion for reviewers and test authors.
M	Create a “Test Data Setup” reusable script that creates the baseline products, locations, and calendars needed for all cases. Reference this script in the pre-condition of each TC.	Reduces duplication and ensures consistent data state.
N	Consider grouping related test cases into logical suites (e.g.,	Improves test planning, execution

Quick Example of an Improved Test-Case Skeleton (illustrative only)

TC-ID	Suite	Priority	Requirement	Pre-condition	Test Steps	Test Data
TC_MP_001	Create-Validate	P1	REQ-RT-101	User Merch Analyst logged in; product `SKU-12345` and location `Store-001` exist.	1. Navigate to Merchandise Plans → Create New. 2. Fill mandatory fields (Plan Name, Calendar, Product, Location, Start/End dates). 3. Click Save.	PlanName=`\${PlanName}` (e.g., "Q3 2026 Plan") Calendar=`\${Calendar}` ...

Only the skeleton is shown; the full test case would retain the detailed steps and validation points.

Bottom line: The current test suite covers the core happy-path functionality but needs stronger structure, data-driven design, explicit verification, and broader negative/RBAC/performance coverage to become a robust, repeatable, and maintainable functional test set for the Merchandise-Plan module. Implementing the suggestions above will raise the quality of the test artefacts and reduce the risk of missed defects in production.

Note : It didn't update the existing test case because the Prompt template has instruction "5. Do NOT update the test case unless requested." And the prompt doesn't explicitly ask for test case update

Output:

Test-Case Review – Merchandise Plan (Retail Domain)

1. Issues Identified

#	Issue	Affected TC(s)	Impact
---	-----	-----	-----

| ****1**** | ****Inconsistent or missing pre-conditions**** – some cases assume data created in earlier tests without explicitly stating that the prerequisite test must have run successfully (e.g., TC-MerchPlan-003, TC-MerchPlan-004, TC-MerchPlan-010-013). | 3, 4, 10-13 | Test execution can become flaky if the dependent test fails or is skipped. |

| ****2**** | ****No clean-up / post-condition steps**** – after creating, publishing, duplicating or deleting plans the environment is left in a changed state. This can affect subsequent runs. | 1-7, 10-13, 15 | Test suite is not repeatable; may require manual reset. |

| ****3**** | ****Test data is hard-coded and not parameterised**** – values such as “Q3 2026 Plan”, SKU-12345, dates, etc., are embedded directly in the steps. This reduces re-usability and makes maintenance harder when data changes. | All | Limits scalability and data-driven execution. |

| ****4**** | ****Ambiguous verification points**** – several “Expected Result” columns describe high-level outcomes but omit concrete UI elements, API responses, or database checks (e.g., TC-MerchPlan-001 “appears in the list”, TC-MerchPlan-012 “Total WP Sales = 245,362”). | 1, 12, 13, 14 | May lead to subjective interpretation and missed defects. |

| ****5**** | ****Missing negative-scenario coverage**** – only a few negative cases are present (duplicate name, duplicate product, delete published plan). Other important validations are absent, such as:

• Invalid product SKU
• Unauthorized user trying to delete a draft
• Attempt to publish without required fields filled. | – | Reduces confidence in robustness of the Merchandise-Plan module. |

| ****6**** | ****Role-based access test limited to a single role**** – TC-MerchPlan-015 checks only the “Viewer” role. It does not verify that a “Merch Analyst” *can* publish, nor does it test other roles (e.g., “Admin”, “Store Manager”). | 15 | Incomplete RBAC validation. |

| ****7**** | ****Date handling edge cases not covered**** – only a simple start-date > end-date check (TC-MerchPlan-009). No tests for leap-year dates, timezone differences, or boundary dates (e.g., same-day start/end). | 9 | Potential date-related bugs remain unchecked. |

| ****8**** | ****Audit-trail verification is superficial**** – TC-MerchPlan-014 only checks creation log. Updates, deletions, and publish actions are not verified. | 14 | Audit compliance gaps may go unnoticed. |

| ****9**** | ****Performance / concurrency not addressed**** – No test validates system behaviour when multiple users edit/duplicate plans simultaneously, or when a large number of products are added to a plan. | – | May miss scalability or locking issues. |

| ****10**** | ****Inconsistent naming / formatting**** – Some TC IDs use hyphens (TC-MerchPlan-001) while others use underscores in description; step numbering mixes HTML `<br>` tags with plain text. This hampers readability and automated parsing. | All | Reduces clarity and tool-compatibility. |

| ****11**** | ****Missing environment/context details**** – No column for “Test Environment” (e.g., DEV, QA, PROD) or “Browser/Device” for UI tests. | All | Execution may vary across environments without traceability. |

| ****12**** | ****No explicit acceptance criteria for “Draft” vs “Published” status**** – Expected results mention status change but do not verify that the status field is read-only, colour-coded, or reflected in API payloads. | 3-4, 6-7 | Ambiguity in what constitutes a successful status transition. |

| ****13**** | ****Lack of traceability to requirements**** – No reference to functional requirement IDs or user stories, making impact analysis difficult. | All | Gaps in requirement-coverage matrix. |

| ****14**** | ****“Actual Result” column left blank**** – While understandable for a design-time document, the template should indicate that this column will be populated during execution, or be omitted from the design artefact. | All | Minor but can cause confusion for reviewers. |

2. Suggestions for Improvement

| # | Recommendation | Rationale / How to Apply |

| --- | ----- | ----- |

| ****A**** | ****Add explicit pre-condition and post-condition sections**** for each test case. ****
e.g., “Pre-condition: Plan ****Q3 2026 Plan**** exists in Draft status.” ****
Post-condition: Delete the created plan to restore baseline.****** | Guarantees test independence and repeatability. |

| ****B**** | ****Parameterise test data**** using a data-driven approach (CSV, Excel, JSON, or test-management tool variables). Replace hard-coded values with placeholders (e.g., `\${PlanName}`, `\${SKU}`). | Simplifies maintenance, enables multiple data sets, and supports CI execution. |

| ****C**** | ****Detail verification steps****: specify UI element identifiers (label, tooltip, CSS selector), API response fields, or database queries. Example: “Verify that the plan appears in the grid with column ****Plan Name = Q3 2026 Plan**** and ****Status = Draft****.” | Removes ambiguity, supports automated validation, and ensures consistent defect reporting. |

| ****D**** | ****Expand negative-scenario coverage****:
• Attempt to create a plan with an invalid SKU.
• Try to publish a plan while missing mandatory fields.
• Delete a Draft plan with a user lacking delete permission. | Improves confidence that validation logic is comprehensive. |

| ****E**** | ****Broaden RBAC tests****: include at least three roles – Viewer (cannot publish/delete), Merch Analyst (can publish, edit Draft), and Admin (full rights). Verify both allowed and denied actions for each role. | Confirms that role-based security is correctly enforced across the matrix. |

| ****F**** | ****Add date-edge-case tests****:
• Start = End (date same day).
• Leap-year date (Feb 29).
• Dates across DST changes or different time zones if applicable. | Prevents subtle bugs related to calendar handling. |

| ****G**** | ****Enhance audit-trail verification****: create separate tests for creation, edit, publish, duplicate, and delete actions, checking timestamps, user IDs, and action types in the audit log. | Supports compliance and traceability requirements. |

| ****H**** | ****Introduce performance / concurrency tests**** (optional for functional suite, but valuable):
• Add 500 products to a plan and verify response time
• Simultaneous duplicate actions by two users and verify locking behavior. | Detects scalability and race-condition issues early. |

| **I** | **Standardise naming and formatting**:
• Use a consistent TC-ID pattern (e.g., `TC\_MP\_001`).
• List steps as numbered bullet points without HTML tags.
• Keep columns aligned and avoid mixed markup. | Improves readability and enables import into test-management tools. |

| **J** | **Add “Environment” and “Browser/Device” columns** (or a global test-environment section) to capture where the test should run. | Provides traceability across environments and helps reproduce defects. |

| **K** | **Link each test case to requirement/user-story IDs** (e.g., `REQ-RT-102`). | Facilitates coverage analysis and impact assessment for changes. |

| **L** | **Clarify the purpose of the “Actual Result” column** in the design document – either remove it (to be filled during execution) or add a note such as “To be populated after test run”. | Prevents confusion for reviewers and test authors. |

| **M** | **Create a “Test Data Setup” reusable script** that creates the baseline products, locations, and calendars needed for all cases. Reference this script in the pre-condition of each TC. | Reduces duplication and ensures consistent data state. |

| **N** | **Consider grouping related test cases into logical suites** (e.g., “Create & Validate”, “Edit & Publish”, “Delete & RBAC”). Add a “Test Suite” column or separate documentation. | Improves test planning, execution ordering, and reporting. |

Quick Example of an Improved Test-Case Skeleton (illustrative only)

| TC-ID | Suite | Priority | Requirement | Pre-condition | Test Steps | Test Data | Expected Result | Post-condition |

|-----|-----|-----|-----|-----|-----|-----|-----|-----|

| TC_MP_001 | Create-Validate | P1 | REQ-RT-101 | User **Merch Analyst** logged in; product `SKU-12345` and location `Store-001` exist. | 1. Navigate to **Merchandise Plans** → Create New*.
2. Fill mandatory fields (Plan Name, Calendar, Product, Location, Start/End dates).
3. Click **Save**. | PlanName=`\${PlanName}` (e.g., “Q3 2026 Plan”)
Calendar=`\${Calendar}`
... | • Success toast “Plan saved”.
• New row appears in the grid with **Plan Name = \${PlanName}**, **Status = Draft**.
• Database record exists with matching values. | Delete the created plan via API to restore baseline. |

Only the skeleton is shown; the full test case would retain the detailed steps and validation points.

****Bottom line:**** The current test suite covers the core happy-path functionality but needs stronger structure, data-driven design, explicit verification, and broader negative/RBAC/performance coverage to become a robust, repeatable, and maintainable functional test set for the Merchandise-Plan module. Implementing the suggestions above will raise the quality of the test artefacts and reduce the risk of missed defects in production.

Scenario 2:

Prompt: Review the test case and make necessary updates

Output Screenshot:

Test-Case Review – Merchandise Plan (Retail)

Below are the findings from the review of the supplied test-case suite, followed by concrete suggestions and a revised version of the test cases that incorporates the improvements.

1. List of Issues

#	Issue	Impact
1	Inconsistent pre-requisite wording – e.g., “Blank form” is used for both “Create New” and “Edit” scenarios, which can be ambiguous for the tester.	May cause the tester to start from the wrong screen, leading to false failures.
2	Missing explicit verification steps – several cases (e.g., TC-003, TC-010) do not verify the effect of the action (status change, total WP sales update).	Results are not fully validated; defects could be missed.
3	Test-data not fully defined – duplicate-name test, seeding test, and role-based test lack concrete data (user IDs, dates, product SKUs).	Reduces repeatability and makes the test non-deterministic.
4	Expected results are vague – messages such as “Error message” or “Audit log shows ...” do not specify exact UI text, timestamps, or log fields.	Increases ambiguity when comparing actual vs. expected outcomes.

4	Expected results are vague – messages such as “Error message” or “Audit log shows ...” do not specify exact UI text, timestamps, or log fields.	Increases ambiguity when comparing actual vs. expected outcomes.
5	No coverage for negative/edge cases – missing tests for: • Seeding when no prior-year plan exists. • Adding a product with invalid quantity or negative sales. • Concurrent edits (optimistic locking).	Gaps in functional coverage; potential defects remain untested.
6	Duplicate or overlapping test IDs – TC-009 and TC-010 both list “Blank form” as pre-requisite, which can be confusing when tracing execution.	Maintenance overhead and risk of executing the wrong test.
7	Lack of environment / data-setup details – No column for “Test Environment” (e.g., DEV, QA) or “Data Setup Script”.	Test execution may vary across environments; reproducibility suffers.
8	Inconsistent terminology – “WP Sales”, “Weekly Planned Sales”, “Total WP Sales” are used interchangeably.	Can lead to misunderstanding of which field is being validated.
9	Missing “Pass/Fail” and “Comments” columns – The current “Actual Result” column is empty, making it hard to capture execution outcome.	Reduces usefulness of the test-case sheet for reporting.
10	No explicit cleanup steps – Tests that create data (e.g., TC-001, TC-005) do not specify how the data will be removed after	Test data can accumulate, affecting subsequent runs.

10	No explicit cleanup steps – Tests that create data (e.g., TC-001, TC-005) do not specify how the data will be removed after execution.	Test data can accumulate, affecting subsequent runs.
11	Formatting of steps – HTML ` ` tags are used inside the table; this hampers readability in many test-management tools.	Poor readability and potential import errors.
12	Priority assignment – Some P2 tests (e.g., duplicate-product check) are arguably critical for data integrity and should be P1.	Mis-prioritisation may affect test planning.
13	Missing localisation / UI-accessibility checks – No test verifies that error messages are displayed in the user's selected language or that UI controls meet accessibility standards.	Non-functional requirements are not validated.

2. Suggestions for Improvement

#	Recommendation	Reason
1	Standardise pre-requisite column – Use explicit statements such as "User is on Create Merchandise Plan screen" or "Plan <PlanName> exists in Draft status".	Removes ambiguity and speeds up test set-up.
2	Add explicit verification steps after each action (e.g., "Verify that the	Guarantees that the expected

2. Suggestions for Improvement

3	Define a reusable Test Data Sheet – Store product SKUs, user credentials, dates, and expected messages in a separate CSV/Excel file and reference them (e.g., 'DataSet: DS001').	Improves maintainability and enables data-driven execution.
4	Specify exact UI messages (including punctuation and case) and log fields (e.g., 'CreatedBy = userID', 'CreatedOn = 2026-04-30 10:15:00').	Enables deterministic pass/fail evaluation.
5	Add missing negative/edge-case tests – e.g., "Seed from Previous Year when no prior plan exists", "Add product with negative sales", "Two users attempt to edit the same plan simultaneously".	Increases functional coverage and reduces risk.
6	Introduce additional columns – 'Test Environment', 'Data Setup Script', 'Cleanup Steps', 'Pass/Fail', 'Comments'.	Provides full context for execution and reporting.
7	Consolidate terminology – Choose a single term (e.g., Weekly Planned Sales (WPS)) and use it consistently across all test cases.	Improves clarity for testers and developers.
8	Re-evaluate priority levels – Move critical data-integrity checks (duplicate product, duplicate name) to P1.	Aligns test effort with business impact.
9	Replace HTML line-breaks with numbered lists – Use a separate "Step" column or bullet points.	Improves readability and compatibility with test-management tools.

9	Replace HTML line-breaks with numbered lists – Use a separate “Step” column or bullet points.	Improves readability and compatibility with test-management tools.
10	Add cleanup actions – e.g., “Delete plan <PlanName> via API after test execution”.	Prevents data pollution across runs.
11	Add localisation / accessibility test – Verify that error messages appear in the selected language and that UI controls meet WCAG 2.1 AA criteria.	Covers non-functional requirements.
12	Create a “Test Case Template” that all future cases follow (Header, Pre-req, Steps, Data, Expected Result, Cleanup, etc.).	Ensures consistency across the suite.

3. Revised Test-Case Suite (Illustrative)

Below is a **re-formatted and enhanced** version of the original suite. Only the most critical changes are shown; the same pattern should be applied to the remaining cases.

TC-ID	Priority	Test Description	Test Environment	Pre-requisite	Test Steps	Test Data
TC-MerchPlan-001	P1	Create a Merchandise	QA-WEB	User RetailAnalyst is	1. Click Create New.	See “Test Data Sheet – DS001”.

Below is a **re-formatted and enhanced** version of the original suite. Only the most critical changes are shown; the same pattern should be applied to the remaining cases.

TC-ID	Priority	Test Description	Test Environment	Pre-requisite	Test Steps	Test Data
TC-MerchPlan-001	P1	Create a Merchandise Plan with all mandatory fields.	QA-WEB	User RetailAnalyst is logged in and has Create Merchandise Plan role. User is on Create Merchandise Plan screen.	1. Click Create New. 2. Fill Plan Name = 'Q3_2026_Plan'. 3. Select Calendar = '2026 Q3'. 4. Add Product = 'SKU-12345'. 5. Select Location = 'Store-001'. 6. Set Start Date = '2026-07-01'. 7. Set End Date = '2026-09-30'. 8. Click Save. 9. Verify that a toast "Plan	See "Test Data Sheet – DS001".

Output:

****Test-Case Review – Merchandise Plan (Retail)****

Below are the findings from the review of the supplied test-case suite, followed by concrete suggestions and a revised version of the test cases that incorporates the improvements.

1. List of Issues

#	Issue	Impact
---	-----	-----

- | 1 | ****Inconsistent pre-requisite wording**** – e.g., “Blank form” is used for both “Create New” and “Edit” scenarios, which can be ambiguous for the tester. | May cause the tester to start from the wrong screen, leading to false failures. |
- | 2 | ****Missing explicit verification steps**** – several cases (e.g., TC-003, TC-010) do not verify the effect of the action (status change, total WP sales update). | Results are not fully validated; defects could be missed. |
- | 3 | ****Test-data not fully defined**** – duplicate-name test, seeding test, and role-based test lack concrete data (user IDs, dates, product SKUs). | Reduces repeatability and makes the test non-deterministic. |
- | 4 | ****Expected results are vague**** – messages such as “Error message” or “Audit log shows ...” do not specify exact UI text, timestamps, or log fields. | Increases ambiguity when comparing actual vs. expected outcomes. |
- | 5 | ****No coverage for negative/edge cases**** – missing tests for:
• Seeding when no prior-year plan exists.
• Adding a product with invalid quantity or negative sales.
• Concurrent edits (optimistic locking). | Gaps in functional coverage; potential defects remain untested. |
- | 6 | ****Duplicate or overlapping test IDs**** – TC-009 and TC-010 both list “Blank form” as pre-requisite, which can be confusing when tracing execution. | Maintenance overhead and risk of executing the wrong test. |
- | 7 | ****Lack of environment / data-setup details**** – No column for “Test Environment” (e.g., DEV, QA) or “Data Setup Script”. | Test execution may vary across environments; reproducibility suffers. |
- | 8 | ****Inconsistent terminology**** – “WP Sales”, “Weekly Planned Sales”, “Total WP Sales” are used interchangeably. | Can lead to misunderstanding of which field is being validated. |
- | 9 | ****Missing “Pass/Fail” and “Comments” columns**** – The current “Actual Result” column is empty, making it hard to capture execution outcome. | Reduces usefulness of the test-case sheet for reporting. |
- | 10 | ****No explicit cleanup steps**** – Tests that create data (e.g., TC-001, TC-005) do not specify how the data will be removed after execution. | Test data can accumulate, affecting subsequent runs. |
- | 11 | ****Formatting of steps**** – HTML `<br>` tags are used inside the table; this hampers readability in many test-management tools. | Poor readability and potential import errors. |
- | 12 | ****Priority assignment**** – Some P2 tests (e.g., duplicate-product check) are arguably critical for data integrity and should be P1. | Mis-prioritisation may affect test planning. |
- | 13 | ****Missing localisation / UI-accessibility checks**** – No test verifies that error messages are displayed in the user’s selected language or that UI controls meet accessibility standards. | Non-functional requirements are not validated. |

2. Suggestions for Improvement

| # | Recommendation | Reason |

| ---|-----|-----|

| 1 | ****Standardise pre-requisite column**** – Use explicit statements such as “User is on ****Create Merchandise Plan**** screen” or “Plan ****\<PlanName\>**** exists in ****Draft**** status”. | Removes ambiguity and speeds up test set-up. |

| 2 | ****Add explicit verification steps**** after each action (e.g., “Verify that the ****Publish**** button is disabled”, “Validate that the ****Total WP Sales**** field equals the sum of all product sales”). | Guarantees that the expected behaviour is actually checked. |

| 3 | ****Define a reusable Test Data Sheet**** – Store product SKUs, user credentials, dates, and expected messages in a separate CSV/Excel file and reference them (e.g., `DataSet: DS001`). | Improves maintainability and enables data-driven execution. |

| 4 | ****Specify exact UI messages**** (including punctuation and case) and log fields (e.g., `CreatedBy = userID`, `CreatedOn = 2026-04-30 10:15:00`). | Enables deterministic pass/fail evaluation. |

| 5 | ****Add missing negative/edge-case tests**** – e.g., “Seed from Previous Year when no prior plan exists”, “Add product with negative sales”, “Two users attempt to edit the same plan simultaneously”. | Increases functional coverage and reduces risk. |

| 6 | ****Introduce additional columns**** – `Test Environment`, `Data Setup Script`, `Cleanup Steps`, `Pass/Fail`, `Comments`. | Provides full context for execution and reporting. |

| 7 | ****Consolidate terminology**** – Choose a single term (e.g., ****Weekly Planned Sales (WPS)****) and use it consistently across all test cases. | Improves clarity for testers and developers. |

| 8 | ****Re-evaluate priority levels**** – Move critical data-integrity checks (duplicate product, duplicate name) to P1. | Aligns test effort with business impact. |

| 9 | ****Replace HTML line-breaks with numbered lists**** – Use a separate “Step” column or bullet points. | Improves readability and compatibility with test-management tools. |

| 10 | ****Add cleanup actions**** – e.g., “Delete plan ****\<PlanName\>**** via API after test execution”. | Prevents data pollution across runs. |

| 11 | ****Add localisation / accessibility test**** – Verify that error messages appear in the selected language and that UI controls meet WCAG 2.1 AA criteria. | Covers non-functional requirements. |

| 12 | ****Create a “Test Case Template”**** that all future cases follow (Header, Pre-req, Steps, Data, Expected Result, Cleanup, etc.). | Ensures consistency across the suite. |

3. Revised Test-Case Suite (Illustrative)

Below is a **re-formatted and enhanced** version of the original suite. Only the most critical changes are shown; the same pattern should be applied to the remaining cases.

| TC-ID | Priority | Test Description | Test Environment | Pre-requisite | Test Steps | Test Data |
Expected Result | Cleanup |

|-----|-----|-----|-----|-----|-----|-----|-----|

| **TC-MerchPlan-001** | P1 | Create a Merchandise Plan with all mandatory fields. | QA-WEB |
User **RetailAnalyst** is logged in and has **Create Merchandise Plan** role. User is on **Create Merchandise Plan** screen. | 1. Click **Create New**.
2. Fill **Plan Name** = **Q3_2026_Plan**.
3. Select **Calendar** = **2026 Q3**.
4. Add **Product** = **SKU-12345**.
5. Select **Location** = **Store-001**.
6. Set **Start Date** = **2026-07-01**.
7. Set **End Date** = **2026-09-30**.
8. Click **Save**.
9. Verify that a toast **"Plan saved successfully."** appears.
10. Verify the plan appears in the list with **Status** = **Draft**. | See "Test Data Sheet – DS001". | • Plan record created with the exact values entered.
• Status = **Draft**.
• Audit log entry: **CreatedBy = RetailAnalyst**, **CreatedOn = <timestamp>**.
• No error messages displayed. | Delete plan **Q3_2026_Plan** via API. |

| **TC-MerchPlan-002** | P1 | Prevent creation of a plan with a duplicate name. | QA-WEB |
Q3_2026_Plan already exists in **Draft** status (created by TC-001). | 1. Click **Create New**.
2. Enter **Plan Name** = **Q3_2026_Plan** (same as existing).
3. Fill remaining mandatory fields with valid data (use DS001 values).
4. Click **Save**.
5. Verify error toast **"Plan name already exists."** is displayed.
6. Verify that no new plan record is created. | DS001 (except Plan Name) | • UI shows exact error text **"Plan name already exists."** (case-sensitive).
• Database contains only one record with name **Q3_2026_Plan**. | No cleanup needed (plan already exists). |

| **TC-MerchPlan-003** | P1 | Publish a plan after seeding data from the previous year. | QA-WEB |
Draft plan Q3_2026_Plan exists (TC-001). Prior-year plan **Q3_2025_Plan** exists with **WP Sales = 245,362**. | 1. Open plan **Q3_2026_Plan**.
2. Click **Seed from Previous Year**.
3. In the confirmation dialog, click **Yes**.
4. Verify that **WP Sales** field is populated with **245,362**.
5. Click **Publish**.
6. Verify that a toast **"Plan published successfully."** appears.
7. Verify **Status** changes to **Published**.
8. Open **Audit Trail** and confirm entry **PublishedBy = RetailAnalyst**, **PublishedOn = <timestamp>**. | Prior-year plan data (see DS002). | • WP Sales = **245,362** after seeding.
• Status = **Published**.
• Audit log contains publish entry with correct user and timestamp. | Delete both **Q3_2026_Plan** and **Q3_2025_Plan** via API. |

| **TC-MerchPlan-004** | P1 | Ensure a published plan cannot be edited. | QA-WEB |
Plan Q3_2026_Plan is in **Published** status (TC-003). | 1. Open plan **Q3_2026_Plan**.
2. Attempt to modify **Plan Name**.
3. Observe UI controls. | – | • All edit fields are **read-only**.
• **Publish** button is disabled.
• Message banner **"Plan is published – editing not allowed."** is displayed.
• No changes are persisted when **Save** is clicked (button hidden). | No cleanup needed (plan remains published). |

| **TC-MerchPlan-005** | P1* | Duplicate a plan with a new name. | QA-WEB |
Draft plan Q3_2026_Plan exists (TC-001). | 1. Open plan **Q3_2026_Plan**.
2. Click **Duplicate**

Scenario 3: Provide input other than test case

Note : Instead of test case, I have provided ICETOP prompt

message empty.

Review the below testcase

✓ Finished in 1.2s

Please provide the test case you would like reviewed so I can analyze it and give you feedback.

Test Case Review Generator

I – Instruction

You are a Test Case Review Engine.

Your task is to analyse the test case and provide feedback.

You must include:

- List of issues
- Suggestions for improvement

CRITICAL RULES (Non-Negotiable):

1. If the input does NOT ask for test case review, respond ONLY with:
"Out of scope: The request does not ask for test case review."
2. Do NOT update the test case until it is explicitly requested.

Output :

✓ Finished in 0.9s

Out of scope: The request does not ask for test case review.